

TP3 - Architecture interne du contrôleur de cache

C. Application logicielle

Question C1 : Quelle est l'adresse de la première instruction de la fonction main ? Quelle est l'adresse de la première instruction de la boucle (instruction correspondant à l'étiquette *loop*) ?

Première instruction de main : D'après le fichier [seg.ld](#) et la structure des segments, le segment `seg_code` démarre à l'adresse 0x0040 0000. La première instruction (`addi $29, $29, -4`) se trouve exactement à cette adresse.

Adresse de la boucle : Le prologue contient 5 instructions (l'instruction `la` compte pour 2 instructions matérielles `lui` et `ori`). Soit $5 \times 4 = 20$ octets. L'étiquette `loop` pointe sur l'adresse 0x0040 0014.

Question C2 : Quelles sont les adresses de base des trois tableaux A, B, C ?

Le segment `seg_data` commence à 0x0100 0000. Dans le `main.s` suivre:

- Un pointeur de 4 octets (`.word main`) puis un espacement de 124 octets (`.space 124`). Le décalage total est de 128 octets (0x80).
- Tableau A commence à 0x0100 0080 et occupe 80 octets.
suivie de `.space 48`
- Tableau B commence à 0x0100 0100 et occupe 80 octets.
suivie de `.space 48`
- Tableau C commence à 0x0100 0180 et occupe 80 octets.

Question C3 : Pourquoi l'instruction `sw` est-elle placée après l'instruction `bne` (test de fin de boucle) ?

L'instruction `sw` est placée juste après `bne` pour exploiter le délai de branchement. On optimise l'exécution du programme.

Question C4 : Quel est le nombre de cycles nécessaires pour exécuter une itération de la boucle, dans l'hypothèse où toutes les instructions et toutes les données sont dans le cache (pas de MISS).

Avec un cache parfait, la boucle contient 7 instructions utiles exécutées par itération. De ce fait, si on veut atteindre un MIPS idéal (CPI = 1), une itération prend 7 cycles.

D. Fonctionnement du cache instruction

Question D1 : Sur combien de bits sont codés les trois champs BYTE, SET, et TAG d'une adresse 32 bits ? Quelles sont leurs valeurs pour l'adresse 0x00400000, correspondant à la première instruction du programme ?

Le cache fait 128 octets avec des lignes de 16 octets.

$128/16 = 8$ lignes

- **BYTE** : 4 bits (bits [3:0])
- **SET** : 3 bits (bits [6:4])
- **TAG** : 25 bits restants (bits [31:7])

Pour l'adresse 0x0040 0000 :

BYTE = 0, SET = 0, TAG = 0x00 8000

Question D2 : Représentez dans le tableau ci-dessous le contenu du cache instruction à la fin de la première itération de la boucle. Quelles instructions ont déclenché un MISS sur le cache instruction pour atteindre cet état ?

TAG	V	word3	word2	word1	word0
0x008000	1	li	la	la	addi
0x008000	1	addi	lw	lw	li
0x008000	1	sw	bne	add	addi

Les instructions qui ont provoqué un miss:

- addi \$29, \$29, -4
- li \$6, 0
- addi \$8, \$8, 4

Question D3 : Quel est le contenu du cache instruction à la fin de la 20e itération ? Calculez approximativement le taux de MISS lors de l'exécution complète du programme (on ne prend pas en compte ce qui se passe pendant la séquence de boot et pendant l'affichage du message).

Le corps de la boucle étant très court (environ 8 instructions, soit 2 lignes de cache), il tient entièrement dans le cache instruction. Le contenu du cache reste donc identique à celui de la fin de la première itération.

Taux de MISS : Les seuls MISS ont lieu lors de la première itération. Pour les 19 itérations suivantes, toutes les instructions feront un HIT (taux de MISS tend vers 0%).

Question D4 : Pour quel type de cache l'état MISS_SELECT est-il indispensable ?

Cet état est indispensable pour les caches associatifs par ensembles ou totalement associatifs. Il sert à exécuter l'algorithme de remplacement pour "sélectionner" quelle voie d'un SET donné doit être écrasée par la nouvelle ligne.

Question D5 : Complétez le graphe de l'automate ICACHE_FSM ci-dessous, en attachant à chaque transition sa condition de franchissement. Les conditions de transition dépendent des signaux Booléens suivants :

A : IREQ. IUNC
B : IREQ. !IUNC. IMISS
C : !IREQ + (IREQ . ! IUNC . ! IMISS)
F : VALID
G : ERROR
H : ! VALID . ! VALID
I : 1
J : ! VALID . ! ERROR
K : ERROR
L : VALID
M : 1
N : 1
O : 1

Question D6 : Dans quel état est forcé cet automate lors de l'activation du signal RESETN ? Quel doit être l'autre effet du signal RESETN sur le cache instruction ?

Lors du RESETN, l'automate est forcé dans l'état IDLE. L'autre effet critique du RESET sur un contrôleur de cache est d'invalider toutes les lignes de cache.

E. Fonctionnement du cache de données

Question E1 : Quelles sont les valeurs des trois champs BYTE, SET, et TAG pour les adresses des éléments A[0] et B[0] des deux tableaux ? Donnez le contenu du cache de données à la fin de la première itération de la boucle, en précisant quelles instructions entraînent un MISS sur le cache de données et donc un gel du processeur.

BYTE a 4 bits, SET a 3 bits, TAG = 25 bits

l'adresse de A[0] : 0x0100 0080

en binaire : 0000 000 10 0000 0000 0000 0001 000 0000

BYTE = 0x0, SET = 0x0 , TAG = 0x20001

l'adresse de B[0] : 0x0100 0100

en binaire : 0000 0001 0000 0000 0000 0001 0000 0000

BYTE = 0x0 , SET = 0x0 , TAG = 0x20002

Question E2 : Calculez le taux de MISS sur le cache de données pour l'exécution complète des 20 itérations de cette boucle. Donnez le contenu du cache de données à la fin de la 20e itération de la boucle.

Les adresses de A[0] et B[0] atterrissent exactement dans la même case (SET 0).

De ce fait, B[i] est lu après A[i] dans la boucle, la ligne de cache de A est systématiquement écrasée par celle de B. A la fin de la première itération, la famille 0 contiendra les 4 premiers mots du tableau B. AVEC TAG 0x020002. Le taux de MISS sur le cache de données en lecture sera bien de 100%

Question E3 : Complétez le graphe de cet automate, en attachant à chaque transition sa condition de franchissement. Ces conditions dépendent des signaux Booléens suivants :

A: DREQ. !WRITE. DUNC

B: DREQ. !WRITE. !DUNC. DMISS

C: !DREQ

D: DREQ. WRITE. !DUNC. !DMISS

E: DREQ. WRITE. (DUNC | DMISS)

F: VALID. !ERROR

G: ERROR

H: !VALID. !ERROR

I: 1

J: !VALID. !ERROR

K: ERROR

L: VALID. !ERROR

M: 1

N: 1

O: 1

P: 1

Q: 1

Question E4 : Les expressions booléennes associées aux transitions de sortie de l'état WRITE_REQ sont très proches des expressions booléennes associées aux transitions de sortie de l'état IDLE. Quelle est la différence ?

Dans l'état WRITE_REQ, si le processeur émet une nouvelle requête mais que le tampon d'écriture est plein (signal WOK inactif), l'automate ne peut pas l'accepter immédiatement et doit figer le processeur jusqu'à ce qu'une place se libère, ce qui diffère du comportement passif de l'état IDLE.

F. Accès au PIBUS

Question F1 : Pourquoi les écritures ont-elles la priorité la plus élevée ? Quel est l'inconvénient de ce choix ?

Cela permet de vider le tampon d'écritures le plus vite possible pour éviter un gel du processeur lors d'un sw. L'inconvénient est que les lectures (MISS critiques bloquant le pipeline) sont retardées si le tampon n'est pas vide.

Question F2 : Quel est le mécanisme utilisé par les deux automates ICACHE_FSM et DCACHE_FSM pour transmettre une requête de lecture vers l'automate PIBUS_FSM ? Comment le serveur signale-t-il aux clients que la requête a été prise en compte ? Dans le cas d'une requête de lecture, comment le serveur signale-t-il au client que les données sont disponibles ?

Les automates ICACHE_FSM et DCACHE_FSM lèvent des signaux de requête dédiés vers l'automate PIBUS_FSM.

Acquittement : le serveur PIBUS_FSM valide la prise en compte en changeant d'état (il quitte l'état IDLE) et, dans de nombreuses implémentations, utilise un signal d'acquittement interne ou un mécanisme d'allocation implicite basé sur la priorité pour indiquer au client que sa transaction est en cours de traitement sur le bus.

Disponibilité des données : Le PIBUS_FSM surveille la réponse de la mémoire cible (le signal ACK du bus). Lorsque la mémoire répond READY, cela signifie qu'un mot valide est présent sur le bus de données. Le PIBUS_FSM lève alors le signal VALID vers l'automate client (ICACHE ou DCACHE) pour lui ordonner d'échantillonner (stocker) la donnée à ce cycle précis.

Question F3 : Pourquoi l'automate PIBUS_FSM n'a-t-il pas besoin de signaler qu'une requête d'écriture transmise par le tampon d'écritures postées s'est effectivement terminée ? Quelle est l'utilité de la réponse dans le cas d'une transaction d'écriture ?

- Le tampon d'écriture gère des écritures postées. Une fois que le processeur a déposé la donnée et l'adresse dans le tampon, il passe à l'instruction suivante. Le système n'a pas besoin de savoir à quel cycle exact la donnée a atteint la mémoire RAM.

- L'automate PIBUS_FSM, en tant que maître sur le bus, doit quand même respecter le protocole matériel.

1. Si la mémoire est lente, elle renvoie wait, l'automate doit maintenir l'adresse et la donnée pendant des cycles supplémentaires.

2. si l'adresse est invalide, la cible renvoie ERROR, l'automate doit pouvoir générer une exception matérielle pour alerter le système d'exploitation d'une faute de segmentation.

Question F4 : Complétez le graphe de l'automate PIBUS_FSM ci-dessous, en attachant à chaque transition sa condition de franchissement. Ces conditions dépendent des signaux suivants :

$X = \text{ROK} + \text{SC}$

$Y = \neg(\text{ROK} + \text{SC}) * (\text{DUNC} + \text{DMISS} + \text{IUNC} + \text{IMISS})$

$Z = \neg(\text{ROK} + \text{SC} + \text{DUNC} + \text{DMISS} + \text{IUNC} + \text{IMISS})$

$B = \text{GNT}$

$B' = \neg \text{GNT}$

$C = 1$

$D = \neg \text{ACK}$

$D' = \text{ACK}$

$E = \text{GNT}$

$E' = \neg \text{GNT}$

$F = \text{LAST}$

$F' = \neg \text{LAST}$

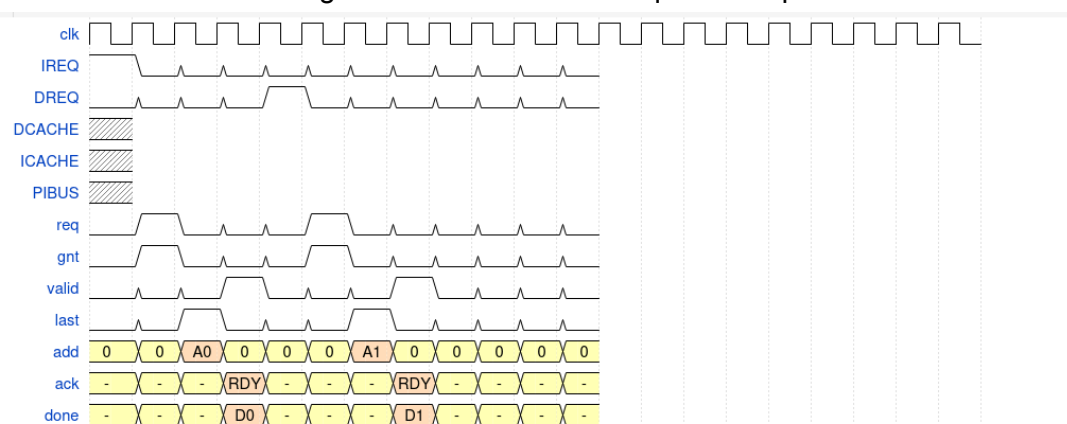
$G = \text{ACK} * \text{LAST}$

$G' = \neg \text{ACK} + \neg \text{LAST}$

$H = \text{ACK}$

$H' = \neg \text{ACK}$

Question F5 : Calculez le coût minimal du miss sur le cache instruction (nombre de cycles de gel du processeur en cas de miss). Pour cela, vous dessinerez explicitement le chronogramme décrivant pour chacun des deux automates, la succession des valeurs stockées dans les registres d'état. Même question pour le cache de données.



A rajouter les les 3 signaux,

Complétez la question F6 et la partie G

Question F6 : Calculez le nombre total de cycles pour exécuter les 20 itérations de la boucle de la section C, en prenant en compte les cycles de gel dûs aux miss sur le cache instruction et sur le cache de données. En déduire la valeur du CPI (nombre moyen de cycles par instruction).

Un MISS coûte 6 cycles de gel.

1. Calcul pour un MIPS idéal :
 - Prologue: 5 instructions -> 5 cycles
 - Boucle: $7 * 20 = 140$ cycles
 - total = 145 cycles
2. Pénalités des MISS instruction :
 - On a vu en D2, le code s'étend sur 3 lignes de cache. Il y a 3 MISS instructions
 - $3 \text{ MISS} * 6 = 18$ cycles de gel.
3. Pénalités des MISS data :
 - On a vu en E1/E2, les tableaux A et B sont en conflit (100% de taux de MISS)
 - $40 \text{ MISS} * 6 = 240$ cycles de gel.

Temps total = $145 + 18 + 240 = 403$ cycles.

Calcul du CPI : $403 / 145 \approx 2.78$ cycles par instructions

G. Expérimentation par simulation

j'ai bien vérifier les adresse et taille des segments dans tp3_top.cpp et seg.ld
j'ai des erreurs a cause du soclib

Message d'erreur :

```
ji@brumel tp3_etudiant]$ soclib-cc -p tp3.desc -t systemcass -o simul.x  
soclib-cc: Entering directory `/users/enseig/ji/MULTI/tp3_etudiant'
```

Traceback (most recent call last):

```
File "/users/outil/soc/soclib/utls/lib/python/soclib_cc_main.py", line 451, in <module>  
    sys.exit(main())
```

```
File "/users/outil/soc/soclib/utls/lib/python/soclib_cc_main.py", line 302, in main  
    return compile_platform(opts.platform, opts)
```

```
File "/users/outil/soc/soclib/utls/lib/python/soclib_cc_main.py", line 394, in compile_platform  
    todo = pf.parse(platform)
```

```
File "/users/soft/soc/soclib/utls/lib/python/sd_parser/platform.py", line 115, in parse  
    exec file(filename) in gbls, locs
```

```
IOError: [Errno 2] No such file or directory: 'tp3.desc'
```

SoCLib-cc failed unexpectedly. To submit a bug report,
please re-run soclib-cc with --bug-report

(i.e run: 'sclib-cc -p tp3.desc -t systemcass -o simul.x --bug-report')